Putting it All Together: IAM Lab (CodeGrade)

(https://github.com/learn-co-curriculum/python-p4-iam-putting-it-all-together-lab) (https://github.com/learn-co-curriculum/python-p4-iam-putting-it-all-together-lab/issues/new)

Learning Goals

- Authenticate a user with a username and password.
- Authorize logged in users for specific actions.

Key Vocab

- **Identity and Access Management (IAM)**: a subfield of software engineering that focuses on users, their attributes, their login information, and the resources that they are allowed to access.
- **Authentication**: proving one's identity to an application in order to access protected information; logging in.
- **Authorization**: allowing or disallowing access to resources based on a user's attributes.
- **Session**: the time between a user logging in and logging out of a web application.
- **Cookie**: data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

This is the biggest lab yet for this phase, so make sure to set aside some time for this one. It's set up with a few different checkpoints so that you can build out the features incrementally. By the end of this lab, you'll have built out full authentication and authorization flow using sessions and cookies in Flask, so getting this lab under your belt will give you some good code to reference when you're building your next project with *auth*. Let's get started!

Setup

As with other labs in this section, there is some starter code in place for a Flask API backend and a React frontend. To get set up, run:

```
$ pipenv install && pipenv shell
$ npm install --prefix client
$ cd server
```

You can work on this lab by running the tests with `pytest`. It will also be helpful to see what's happening during the request/response cycle by running the app in the browser. You can run the Flask server with:

```
$ python app.py
```

Note that running `python app.py` will generate an error if you haven't created your models and run your migrations yet.

And you can run React in another terminal from the project root directory with:

```
$ npm start --prefix client
```

Models

Create a `User` model with the following attributes:

- `id` that is an integer type and a primary key.
- `username` that is a `String` type.
- `_password_hash` that is a `String` type.
- `image_url` that is a `String` type.
- `bio` that is a `String` type.

Your `User` model should also:

- incorporate `bcrypt` to create a secure password. Attempts to access the `password_hash` should be met with an `AttributeError`.
- constrain the user's username to be **present** and **unique** (no two users can have the same username).
- **have many** recipes.

Next, create a `Recipe` model with the following attributes:

- a recipe **belongs to** a user.
- `id` that is an integer type and a primary key.
- `title` that is a `String` type.
- `instructions` that is a `String` type.
- `minutes_to_complete` that is an `Integer` type.

Add database constraints for the `Recipe` model:

Your `Recipe` model should also:

- constrain the `title` to be present.
- constrain the `instructions` to be present and at least 50 characters long.

Run the migrations after creating your models. You'll need to run `flask db init` before running `flask db revision autogenerate` or `flask db upgrade`.

Ensure that the tests for the models are passing before moving forward. To run the tests for *only* the model files, run:

```
$ pytest testing/models_testing/
```

Once your tests are passing, you can seed your database from within the `server` directory by running:

```
$ python seed.py
```

Sign Up Feature

After creating the models, the next step is building out a sign up feature.

Handle sign up by implementing a `POST /signup` route. It should:

- Be handled in a `Signup` resource with a `post()` method.
- In the `post()` method, if the user is valid:
  - Save a new user to the database with their username, encrypted password, image URL, and bio.
  - Save the user's ID in the session object as `user_id`.
  - Return a JSON response with the user's ID, username, image URL, and bio; and an HTTP status code of 201 (Created).
- If the user is not valid:
  - Return a JSON response with the error message, and an HTTP status code of 422 (Unprocessable Entity).

> **Note: Recall that we need to format our error messages in a way that makes it easy to display the information in our frontend. For this lab, because we are setting up multiple validations on our `User` and `Recipe` models, our error responses need to be formatted in a way that accommodates multiple errors.**

Auto-Login Feature

Users can log into our app! 🎉 But we want them to **stay** logged in when they refresh the page, or navigate back to our site from somewhere else.

Handle auto-login by implementing a `GET /check_session` route. It should:

- Be handled in a `CheckSession` resource with a `get()` method.
- In the `get()` method, if the user is logged in (if their `user_id` is in the session object):
  - Return a JSON response with the user's ID, username, image URL, and bio; and an HTTP status code of 200 (Success).
- If the user is **not** logged in when they make the request:
  - Return a JSON response with an error message, and a status of 401 (Unauthorized).

Make sure the signup and auto-login features work as intended before moving forward. You can test the `CheckSession` requests with pytest:

```
$ pytest testing/app_testing/app_test.py::TestCheckSession
```

You should also be able to test this in the React application by signing up via the sign up form to check the `POST /signup` route; and refreshing the page after logging in, and seeing that you are still logged in to test the `GET /check_session` route.

Login Feature

Now that users can create accounts via the API, let's give them a way to log back into an existing account.

Handle login by implementing a `POST /login` route. It should:

- Be handled in a `Login` resource with a `post()` method.
- In the `post()` method, if the user's username and password are authenticated:
  - Save the user's ID in the session object.
  - Return a JSON response with the user's ID, username, image URL, and bio.
- If the user's username and password are not authenticated:
  - Return a JSON response with an error message, and a status of 401 (Unauthorized).

Make sure this route works as intended by running `pytest testing/app_testing/app_test.py::TestLogin` before moving forward. You should also be able to test this in the React application by logging in via the login form.

Logout Feature

Users can log into our app! 🎉 Now, let's give them a way to log out.

Handle logout by implementing a `DELETE /logout` route. It should:

- Be handled in a `Logout` resource with a `delete()` method.
- In the `delete()` method, if the user is logged in (if their `user_id` is in the session object):
  - Remove the user's ID from the session object.
  - Return an empty response with an HTTP status code of 204 (No Content).
- If the user is **not** logged in when they make the request:
  - Return a JSON response with an error message, and a status of 401 (Unauthorized).

Make sure the login and logout features work as intended before moving forward. You can test the `Logout` requests with RSpec:

```
$ pytest testing/app_testing/app_test.py::TestLogout
```

You should also be able to test this in the React application by logging in to check the `POST /login` route; and logging out with the logout button to test the `DELETE /logout` route.

Recipe List Feature

Users should only be able to view recipes on our site after logging in.

Handle recipe viewing by implementing a `GET /recipes` route. It should:

- Be handled in a `RecipeIndex` resource with a `get()` method
- In the `get()` method, if the user is logged in (if their `user_id` is in the session object):
  - Return a JSON response with an array of all recipes with their title, instructions, and minutes to complete data along with a nested user object; and an HTTP status code of 200 (Success).
- If the user is **not** logged in when they make the request:
  - Return a JSON response with an error message, and a status of 401 (Unauthorized).

Recipe Creation Feature

Now that users can log in, let's allow them to create new recipes!

Handle recipe creation by implementing a `POST /recipes` route. It should:

- Be handled in the `RecipeIndex` resource with a `post()` method.
- In the `post()` method, if the user is logged in (if their `user_id` is in the session object):
  - Save a new recipe to the database if it is valid. The recipe should **belong to** the logged in user, and should have title, instructions, and minutes to complete data provided from the request JSON.
  - Return a JSON response with the title, instructions, and minutes to complete data along with a nested user object; and an HTTP status code of 201 (Created).
- If the user is **not** logged in when they make the request:
  - Return a JSON response with an error message, and a status of 401 (Unauthorized).

- If the recipe is **not valid**:
  - Return a JSON response with the error messages, and an HTTP status code of 422 (Unprocessable Entity).

After finishing the `RecipeIndex` resource, you're done! Make sure to check your work. You should be able to run the full test suite now with `pytest`.

You should also be able to test this in the React application by creating a new recipe with the recipe form, and viewing a list of recipes.

Resources

- **API - Flask: `class flask.session` (https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)**
- **User's Guide - Flask RESTful (https://flask-restful.readthedocs.io/en/latest/)**
- **Flask-Bcrypt (https://flask-bcrypt.readthedocs.io/en/1.0.1/)**

This tool needs to be loaded in a new browser window

Load Putting it All Together: IAM Lab (CodeGrade) in a new window